

TP05 — Traversal, upload, commande et SSRF sans sortie du lab

Séance : S09 · Durée : 180 min · Compétence : C07 · Niveau : intermédiaire

Contrat de sécurité

Toutes les expériences restent dans le conteneur ShopLab. L'endpoint SSRF est un simulateur et n'effectue aucune requête réseau. N'utilisez jamais une IP publique, un nom réel ou les métadonnées cloud. Arrêtez si une commande proposée exécute un shell.

Objectifs, prérequis et livrables

Vous allez séparer quatre familles souvent confondues, prouver leur cause et construire des allowlists/canonicalisations. Prérequis : TP04. Livrables : arbre de décision, quatre tests avant/après, politique d'upload, tests unitaires du starter, preuves et cleanup. Réussite : aucune lecture hors racine, aucun nom contrôlé utilisé comme chemin, type/taille vérifiés, aucune commande shell, destination SSRF refusée par allowlist.

Mise en place (15 min)

```
cd 07_public_student_export/labs/shoplab
./scripts/reset.sh
base=http://127.0.0.1:${SHOPLAB_HTTP_PORT:-8080}
mkdir -p preuves/TP05
```

A — Traversal (30 min)

Comparez `name=welcome.txt` et `name=../sample-training-vault/demo.txt` via `--data-urlencode`. En mode vulnérable, observez seulement le fichier synthétique prévu; ne variez pas la cible. En corrected, attendu 400. Expliquez pourquoi supprimer `../` une seule fois est fragile et proposez : nom de base, résolution canonique, contrôle d'appartenance et identifiant serveur.

B — Upload (35 min)

L'API pédagogique reçoit JSON `{filename, content_type, content_b64}` et écrit seulement dans le volume jetable. Testez : `note.txt`, `../escape.txt`, double extension, type non autorisé, contenu > limite. En corrected, seuls `.txt/.png`, taille ≤64 Kio, nom généré serveur et métadonnées séparées sont acceptés. Prouvez que le fichier n'est pas servi comme HTML exécutable.

C — Command injection (30 min)

Le starter `TP05/safe_report.py` contient une fonction de construction de commande, sans exécution. Montrez pourquoi une chaîne shell contenant un nom utilisateur est ambiguë. Remplacez-la par une liste d'arguments stricte et une allowlist d'identifiants. Les tests doivent couvrir `daily`, `daily;id`, espace, option `--help` et Unicode. N'ajoutez jamais `shell=True`.

D — SSRF (35 min)

Interrogez `/api/url-check` avec : URL malformée, `http://127.0.0.1`, `http://db:5432`, une IP privée documentaire et `https://service.invalid/path`. Aucune n'est réellement fetchée (`fetched=false`). En corrected, l'allowlist de démonstration n'autorise que `https://status.shoplab.invalid/health`. Discutez DNS rebinding, redirections, schémas, ports, résolution IP et egress réseau.

E — Retest croisé (20 min)

Pour chaque famille, documentez : source contrôlée, sink dangereux, frontière, contrôle primaire, contrôle secondaire, résultat fonctionnel légitime. Exécutez les tests du starter et `verify-lab.sh files`.

Vérification, reset et cleanup (15 min)

```
./scripts/verify-lab.sh files
./scripts/reset.sh
./scripts/cleanup.sh
```

Dépannage

Un 404 traversal peut signifier que votre encodage a été normalisé par le client; utilisez `--data-urlencode`. Base64 doit être produit depuis une donnée synthétique. Un résultat `allowed=false` n'est utile que si `fetched=false`. Sur Windows, évitez les chemins hôte dans les payloads : l'exercice cible le chemin Linux interne au conteneur.

Barème /20

Critère	Insuffisant	Conforme	Maîtrisé	Pts
Distinction des risques	mélange	4 sources/sinks	frontières explicites	4
Politiques	blacklist	allowlists + canonique	limites/TOCTOU/egress	7
Tests	cas nominal	négatifs et légitime	propriétés/invariants	6
Sécurité opératoire	cible externe/shell	simulation locale	preuves expurgées/cleanup	3